

МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

**Лабораторная работа № 2 по курсу
«Операционные системы и архитектура компьютера»**

Выполнил: Е. И. Михайлов
Группа: М8О-204БВ-25
Преподаватель: Н. А. Гапонов
Дата: 08.10.2026
Оценка: _____

Москва, 2026

Условие

Цель работы: приобретение практических навыков в управлении процессами в операционной системе и в организации обмена данными между процессами посредством каналов (pipe).

Задание: составить и отладить программу на языке Си, осуществляющую работу с процессами и взаимодействие между ними. Основной процесс должен создать для решения задачи один или несколько дочерних процессов; взаимодействие осуществляется через каналы и/или системные события. Необходимо обрабатывать системные ошибки, которые могут возникнуть в ходе работы. Родительский и дочерние процессы должны быть представлены разными программами. Работа выполнена для операционной системы Windows.

Группа вариантов 3: родительский процесс создаёт два дочерних процесса, child1 и child2, которые можно соединить между собой дополнительным каналом. Родительский процесс принимает от пользователя строки произвольной длины и пересылает их в pipe1. Процессы child1 и child2 производят работу над строками. Child2 пересылает результат своей работы родительскому процессу, а тот выводит полученный результат в стандартный поток вывода.

Вариант 14: child1 переводит строки в нижний регистр; child2 убирает все задвоенные пробелы.

Метод решения

Решение состоит из трёх независимых программ: parent.exe, child1.exe и child2.exe. Родитель создаёт три анонимных канала и образует из дочерних процессов конвейер, аналогичный конструкции `parent | child1 | child2` в командной строке, с тем отличием, что результат возвращается обратно в родителя, а не в консоль напрямую:

- 1) **pipe1** (parent → child1): родитель записывает в него введённую пользователем строку, а child1 читает её как свой стандартный ввод;
- 2) **pipe_mid** (child1 → child2): дополнительный канал, соединяющий дочерние процессы; стандартный вывод child1 является стандартным входом child2;
- 3) **pipe2** (child2 → parent): child2 пишет результат в стандартный вывод, а родитель читает его и выводит в консоль.

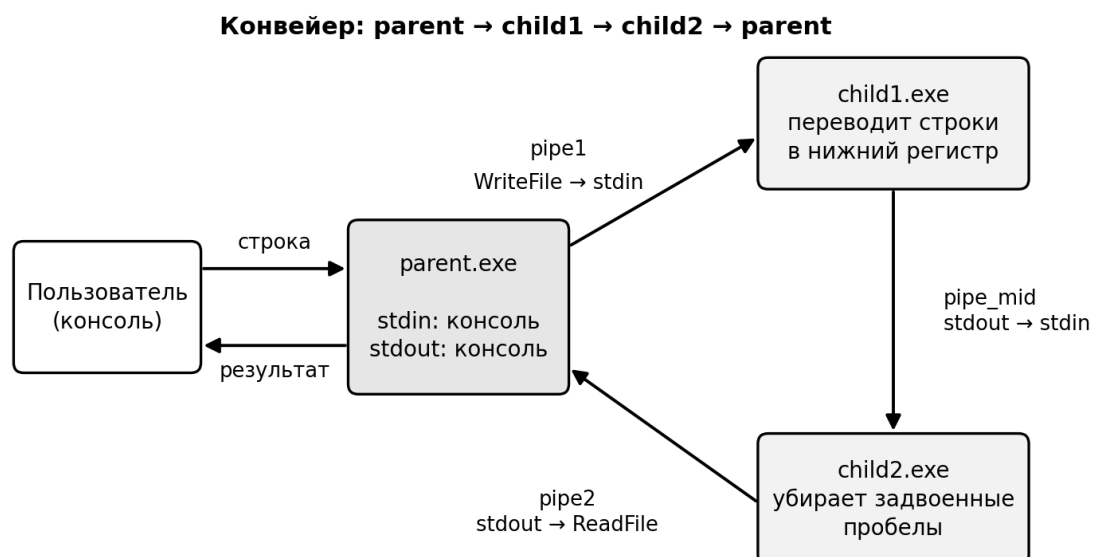


Рисунок 1 – Схема взаимодействия процессов

Дочерние процессы ничего не знают о каналах: каждый из них является обычным фильтром, который читает данные из стандартного ввода и пишет результат в

стандартный вывод. Перенаправление потоков выполняет родитель при вызове `CreateProcessA`: в структуре `STARTUPINFOA` он задаёт нужные дескрипторы в полях `hStdInput` и `hStdOutput` и флаг `STARTF_USESTDHANDLES`. Благодаря этому каждый дочерний процесс можно запустить и проверить отдельно.

Алгоритм родительского процесса:

- 1) создать три канала с наследуемыми дескрипторами; для концов, которые остаются только у родителя (запись в `pipe1` и чтение из `pipe2`), снять признак наследования;
- 2) запустить `child1` (`stdin = pipe1`, `stdout = pipe_mid`), затем закрыть в родителе `hPipe1Read` и `hPipeMidWrite` и только после этого запустить `child2` (`stdin = pipe_mid`, `stdout = pipe2`); порядок важен, см. раздел «Особенности реализации»;
- 3) закрыть оставшиеся ненужные родителю концы (`hPipeMidRead`, `hPipe2Write`);
- 4) в цикле: прочитать строку с консоли, записать её в `pipe1`, прочитать ответ из `pipe2` и вывести его в консоль; пустые строки пропускаются;
- 5) при конце ввода закрыть `hPipe1Write` и `hPipe2Read`, дождаться завершения дочерних процессов (`WaitForSingleObject`) и освободить дескрипторы.

Дочерние процессы работают в цикле «прочитать порцию байт – обработать – записать»: `child1` заменяет каждый символ на его строчный вариант, `child2` пропускает пробел, если предыдущий записанный символ тоже был пробелом. Признак «предыдущий символ – пробел» хранится в переменной, которая живёт между порциями, поэтому последовательность пробелов, разорванная границей порции, тоже схлопывается корректно. Цикл заканчивается, когда `ReadFile` возвращает ошибку или ноль байт – то есть когда все записывающие концы канала закрыты.

Обмен синхронный: родитель после каждой отправленной порции ждёт ответ. Это гарантирует, что результат обработки строки выводится до приглашения к вводу следующей.

Источники: документация Microsoft Learn – «Creating a Child Process with Redirected Input and Output» (learn.microsoft.com/en-us/windows/win32/procthread/creating-a-child-process-with-redirected-input-and-output), справочник по функциям `CreatePipe`, `CreateProcess`, `SetHandleInformation` (learn.microsoft.com/en-us/windows/win32/api), документация `Process Monitor` (learn.microsoft.com/en-us/sysinternals/downloads/procmon).

Описание программы

Проект состоит из трёх программ на языке Си и сценария сборки CMake. Сборка выполнена в конфигурации Debug (в трассе Process Monitor видны отладочные библиотеки `ucrtbased.dll` и `vcruntime140d.dll`), исполняемые файлы создаются в каталоге `build\bin\Debug\`. Родитель запускает потомков по относительным путям `.\child1.exe` и `.\child2.exe`, поэтому `parent.exe` нужно запускать из каталога, в котором лежат все три exe-файла. Программы используют Windows API; полный исходный код приведён в приложении А, в этом разделе цитируются только ключевые фрагменты.

Структура файлов

Таблица 1 – Файлы проекта

Файл	Назначение
<code>parent.c</code>	родительский процесс: создаёт три канала, запускает <code>child1</code> и <code>child2</code> с перенаправлением стандартных потоков, читает строки с консоли, отправляет их в <code>pipe1</code> и выводит ответ из <code>pipe2</code>
<code>child1.c</code>	первый дочерний процесс: читает данные из стандартного ввода, переводит символы в нижний регистр и пишет в стандартный вывод

Файл	Назначение
child2.c	второй дочерний процесс: читает данные из стандартного ввода, удаляет задвоенные пробелы и пишет в стандартный вывод
CMakeLists.txt	сборка трёх исполняемых файлов в общий каталог bin/, чтобы parent.exe находил child1.exe и child2.exe рядом с собой; child1 и child2 собираются раньше parent

Основные типы данных и функции

Все три программы содержат единственную функцию main. Используются следующие типы Windows API и константы:

- HANDLE – дескриптор объекта ядра (конец канала, стандартный поток, процесс);
- SECURITY_ATTRIBUTES – параметры безопасности; поле bInheritHandle = TRUE делает создаваемые дескрипторы наследуемыми;
- STARTUPINFOA – параметры запуска процесса; поля hStdInput, hStdOutput, hStdError задают стандартные потоки потомка, флаг STARTF_USESTDHANDLES включает их использование;
- PROCESS_INFORMATION – результат CreateProcessA: дескрипторы процесса и главного потока и их идентификаторы;
- DWORD – 32-битное беззнаковое целое для счётчиков прочитанных и записанных байт;
- BUFFER_SIZE = 4096 – размер буферов чтения и записи во всех программах.

Роли переменных родителя показаны в таблице 2.

Таблица 2 – Дескрипторы каналов в parent.c

Дескриптор	Канал	Назначение
hPipe1Read	pipe1	читающий конец; передаётся child1 как стандартный ввод, закрывается в родителе сразу после запуска child1
hPipe1Write	pipe1	записывающий конец; остаётся у родителя (наследование отключено)
hPipeMidRead	pipe_mid	читающий конец; передаётся child2 как стандартный ввод
hPipeMidWrite	pipe_mid	записывающий конец; передаётся child1 как стандартный вывод, закрывается в родителе сразу после запуска child1 (до запуска child2)
hPipe2Read	pipe2	читающий конец; остаётся у родителя (наследование отключено)
hPipe2Write	pipe2	записывающий конец; передаётся child2 как стандартный вывод

Обработка ошибок: родитель проверяет результат CreatePipe, CreateProcessA, WriteFile и ReadFile. Если не удалось создать каналы или запустить процесс, программа завершается с кодом 1; ошибка записи или чтения прерывает цикл обмена, после чего выполняется штатная очистка. Дочерние процессы завершают цикл, когда ReadFile сообщает об ошибке или ноль прочитанных байт.

Используемые системные вызовы

Программа использует функции Win32 API, являющиеся обёртками над системными вызовами ядра Windows (например, CreateProcessA выполняет NtCreateUserProcess). Дополнительно применяются функции стандартной библиотеки Си: fgets, printf, fflush, strlen (родитель) и tolower (child1).

Таблица 3 – Используемые системные вызовы

Системный вызов	Назначение в программе
CreatePipe	создание анонимного канала; три вызова в родителе (pipe1, pipe_mid, pipe2)
SetHandleInformation	снятие флага HANDLE_FLAG_INHERIT с концов hPipe1Write и hPipe2Read, чтобы дочерние процессы не получили их копии
GetStdHandle	получение дескрипторов стандартного ввода/вывода/ошибок: в дочерних процессах – для чтения и записи в каналы, в родителе – для передачи stderr потомкам
CreateProcessA	создание дочернего процесса из отдельного exe-файла; bInheritHandles = TRUE и STARTF_USESTDHANDLES обеспечивают перенаправление стандартных потоков
WriteFile	запись данных в канал: родитель – в pipe1, child1 – в pipe_mid, child2 – в pipe2
ReadFile	блокирующее чтение из канала: родитель – из pipe2, child1 – из pipe1, child2 – из pipe_mid; ошибка или 0 байт означают закрытие канала
WaitForSingleObject	ожидание завершения child1 и child2 при выходе из программы
CloseHandle	закрытие ненужных концов каналов и дескрипторов процессов и потоков

Ключевые фрагменты кода

Родитель создаёт три канала. Параметр bInheritHandle = TRUE делает дескрипторы наследуемыми; затем у концов, которые нужны только родителю, наследование отключается.

Листинг 1 – Создание каналов (parent.c)

```
SECURITY_ATTRIBUTES sa;  
sa.nLength = sizeof(SECURITY_ATTRIBUTES);  
sa.bInheritHandle = TRUE;  
sa.lpSecurityDescriptor = NULL;  
  
if (!CreatePipe(&hPipe1Read, &hPipe1Write, &sa, 0) ||  
    !CreatePipe(&hPipeMidRead, &hPipeMidWrite, &sa, 0) ||  
    !CreatePipe(&hPipe2Read, &hPipe2Write, &sa, 0)) {  
    return 1;  
}  
  
SetHandleInformation(hPipe1Write, HANDLE_FLAG_INHERIT, 0);  
SetHandleInformation(hPipe2Read, HANDLE_FLAG_INHERIT, 0);
```

Запуск child1: в STARTUPINFOA указываются дескрипторы, которые потомок получит в качестве стандартного ввода (pipe1) и вывода (pipe_mid). Child2 запускается аналогично со стандартным вводом из pipe_mid и выводом в pipe2.

Листинг 2 – Запуск дочернего процесса с перенаправлением потоков (parent.c)

```
STARTUPINFOA siChild1;
PROCESS_INFORMATION piChild1;
ZeroMemory(&siChild1, sizeof(siChild1));
ZeroMemory(&piChild1, sizeof(piChild1));
siChild1.cb = sizeof(siChild1);
siChild1.hStdInput = hPipe1Read;
siChild1.hStdOutput = hPipeMidWrite;
siChild1.hStdError = GetStdHandle(STD_ERROR_HANDLE);
siChild1.dwFlags |= STARTF_USESTDHANDLES;

char cmdChild1[] = ".\\child1.exe";
if (!CreateProcessA(NULL, cmdChild1, NULL, NULL, TRUE, 0, NULL, NULL, &siChild1,
&piChild1)) {
    return 1;
}
```

Сразу после запуска child1 родитель закрывает свои копии hPipe1Read и hPipeMidWrite – до запуска child2, чтобы тот не унаследовал записывающий конец pipe_mid. Остальные ненужные концы закрываются после запуска child2.

Листинг 3 – Порядок закрытия дескрипторов (parent.c)

```
CloseHandle(hPipe1Read);
CloseHandle(hPipeMidWrite);

CloseHandle(hPipeMidRead);
CloseHandle(hPipe2Write);
```

Основной цикл родителя: чтение строки, отправка в pipe1 и получение результата из pipe2. Пустые строки пропускаются, чтобы не ожидать ответа, которого не будет; ошибка любого вызова прерывает цикл.

Листинг 4 – Цикл обмена данными (parent.c)

```
while (1) {
    fflush(stdout);

    if (!fgets(input, sizeof(input), stdin)) {
        break;
    }

    DWORD len = (DWORD)strlen(input);
    if (len == 0 || (len == 1 && input[0] == '\\n')) {
        continue;
    }

    if (!WriteFile(hPipe1Write, input, len, &bytesWritten, NULL)) {
        break;
    }

    char output[BUFFER_SIZE];
```

```

        if (ReadFile(hPipe2Read, output, sizeof(output) - 1, &bytesRead, NULL) &&
bytesRead > 0) {
            output[bytesRead] = '\\0';
            printf("%s", output);
            fflush(stdout);
        } else {
            break;
        }
    }
}

```

Child1 читает порцию из канала, переводит каждый байт в нижний регистр и пишет её в стандартный вывод. Цикл заканчивается, когда чтение завершилось ошибкой или вернуло 0 байт.

Листинг 5 – Приведение к нижнему регистру (child1.c)

```

while (ReadFile(hStdin, buffer, BUFFER_SIZE, &bytesRead, NULL) && bytesRead > 0) {
    for (DWORD i = 0; i < bytesRead; i++) {
        buffer[i] = (char)tolower((unsigned char)buffer[i]);
    }

    WriteFile(hStdout, buffer, bytesRead, &bytesWritten, NULL);
}

```

Child2 копирует байты в выходной буфер, пропуская пробел, если предыдущий записанный символ тоже был пробелом. Переменная prevWasSpace объявлена вне цикла чтения и сохраняет состояние между порциями.

Листинг 6 – Удаление задвоенных пробелов (child2.c)

```

while (ReadFile(hStdin, inBuffer, BUFFER_SIZE, &bytesRead, NULL) && bytesRead > 0) {
    DWORD outIdx = 0;

    for (DWORD i = 0; i < bytesRead; i++) {
        char c = inBuffer[i];
        if (c == ' ') {
            if (!prevWasSpace) {
                // запись в результат
                outBuffer[outIdx++] = c;
                prevWasSpace = true;
            }
        } else {
            outBuffer[outIdx++] = c;
            prevWasSpace = false;
        }
    }
    if (outIdx > 0) {
        WriteFile(hStdout, outBuffer, outIdx, &bytesWritten, NULL);
    }
}
}

```

Завершение работы родителя: закрытие своих концов каналов и ожидание дочерних процессов.

Листинг 7 – Завершение и очистка (parent.c)

```

CloseHandle(hPipe1Write);
CloseHandle(hPipe2Read);

WaitForSingleObject(piChild1.hProcess, INFINITE);
WaitForSingleObject(piChild2.hProcess, INFINITE);

CloseHandle(piChild1.hProcess);
CloseHandle(piChild1.hThread);
CloseHandle(piChild2.hProcess);
CloseHandle(piChild2.hThread);

```

Результаты

Программа собирается с помощью CMake; parent.exe запускается из каталога сборки (build\bin\Debug\), где рядом лежат child1.exe и child2.exe (потомки запускаются по относительным путям .\child1.exe и .\child2.exe). Ниже показан сеанс работы: пользователь вводит строки, а родитель выводит результат, прошедший через child1 и child2.

Листинг 8 – Пример сеанса работы

```

C:\web\OSI\lab2\build\bin\Debug> parent.exe
HELLO  World  FOO          <-- ввод пользователя
hello world foo           <-- результат (child1 -> child2 -> parent)
HeLlO   Again            <-- ввод пользователя
hello again               <-- результат
    Leading  AND trailing
leading and trailing

```

Результаты проверки на различных входных данных приведены в таблице 4 (пробелы в строках показаны внутри кавычек).

Таблица 4 – Тестовые сценарии

№	Ввод	Вывод	Что проверяется
1	«HELLO World FOO»	«hello world foo»	нижний регистр и схлопывание нескольких пробелов
2	«HeLlO Again»	«hello again»	смешанный регистр, пять пробелов подряд
3	« Leading AND trailing »	« leading and trailing »	пробелы в начале и конце не удаляются, а сводятся к одному
4	«Tab<TAB><TAB>Here X»	«tab<TAB><TAB>here x»	табуляция не считается пробелом и сохраняется
5	пустая строка (Enter)	вывода нет	родитель пропускает пустые строки
6	«ПРИВЕТ Мир ABC»	«ПРИВЕТ Мир abc»	пробелы схлопываются, регистр кириллицы не меняется
7	строка из 6003 символов: 3000×«А», 3 пробела, 3000×«В»	3000×«а», 1 пробел, 3000×«b»	строка длиннее буфера (4096 байт) обрабатывается порциями

Особенности реализации и ограничения

- **Наследование дескрипторов и порядок закрытия.** `CreateProcessA` вызывается с `bInheritHandles = TRUE`, поэтому потомок получает копии всех наследуемых дескрипторов родителя. Если бы родитель держал `hPipeMidWrite` в момент запуска `child2`, тот получил бы копию записывающего конца `pipe_mid` и никогда не увидел бы конец данных на своём вводе. Поэтому `hPipe1Read` и `hPipeMidWrite` закрываются сразу после запуска `child1` (листинг 3). Конец ввода (`Ctrl+Z`, `Enter`) тогда распространяется по цепочке: родитель закрывает `pipe1`, `child1` получает конец данных и завершается, закрывается `pipe_mid`, завершается `child2`, и `WaitForSingleObject` в родителе возвращается. Копии `hPipeMidRead` и `hPipe2Write`, оставшиеся у `child1`, безвредны, так как он завершается раньше родителя. Завершение по `Ctrl+C` также работает: сигнал получают все три процесса консоли (см. раздел о `Process Monitor`).
- **Размер буфера.** Строки длиннее `BUFFER_SIZE` (4096 байт) обрабатываются порциями: результат остаётся корректным (тест 7), но родитель читает ответ из `pipe2` одним вызовом `ReadFile` на каждую отправленную порцию.
- **Регистр и пробелы.** Функция `tolower` в локали «С» преобразует только символы ASCII, поэтому кириллица не меняет регистр. Удаляются только символы пробела (код `0x20`); табуляция и другие пробельные символы сохраняются.
- **Диагностика ошибок.** При сбое системных вызовов программы завершаются с кодом возврата или прерывают цикл, но текст ошибки (`GetLastError`) не выводится.

Демонстрация работы Process Monitor

Лабораторная работа выполнена в Windows, где утилита `strace` недоступна, поэтому вместо неё использован `Process Monitor` из набора `Sysinternals`. Он регистрирует события файловой системы, реестра, процессов и потоков. В отличие от `strace`, он показывает не все системные вызовы: например, вызов `CreateProcessA` отображается как событие «Process Create». Программа `parent.exe` была запущена из каталога `C:\web\OSI\lab2\build\bin\Debug\` (Parent PID 2364 – процесс, из которого она запущена) и остановлена нажатием `Ctrl+C`; запись длилась около 13 секунд (8:50:43.388 – 8:50:56.474). В приложении Б приведены события, относящиеся к жизненному циклу процессов (без обращений к реестру, служебных запросов к файлам и дампов переменных окружения).

Таблица 5 – Хронология основных событий

Время	Процесс (PID)	Событие	Комментарий
8:50:43.388	parent.exe (1076)	Process Start	запуск родителя; Parent PID 2364 – процесс, из которого он запущен
8:50:43.408	parent.exe → child1.exe (7788)	Process Create	первый вызов <code>CreateProcessA</code> , командная строка <code>.\child1.exe</code>
8:50:43.421	parent.exe → child2.exe (11024)	Process Create	второй вызов <code>CreateProcessA</code> , через $\approx 12,6$ мс после первого
8:50:43.43 – 8:50:56.43	все три	нет событий	процессы ожидают ввода (блокирующие <code>ReadFile</code>), ≈ 13 с
8:50:56.431	все три	Thread Create	в каждом процессе создан поток – обработка <code>Ctrl+C</code>
8:50:56.432 – .474	child2, parent, child1	Process Exit	все три процесса завершились с кодом <code>-1073741510</code> (<code>0xC000013A</code>)

Запуск процессов. Родитель запускается в C:\web\OSI\lab2\build\bin\Debug\. Перед созданием каждого потомка он открывает файл child1.exe или child2.exe и отображает его в память (CreateFileMapping с PAGE_EXECUTE); результат «FILE LOCKED WITH ONLY READERS» – штатная информационная запись при создании секции образа. Затем следует событие Process Create и событие Process Start уже нового процесса. В командной строке потомка указан относительный путь .\child1.exe (так он задан в parent.c); он разрешается относительно текущего каталога, поэтому parent.exe нужно запускать из каталога с exe-файлами.

Листинг 9 – Создание child1 (фрагмент трассы Process Monitor, сокращённая запись)

```
8:50:43.3880178 parent.exe 1076 Process Start
    Parent PID: 2364,
    Command line: "C:\web\OSI\lab2\build\bin\Debug\parent.exe",
    Current directory: C:\web\OSI\lab2\build\bin\Debug\
8:50:43.4007867 parent.exe 1076 CreateFileMapping ...\.Debug\child1.exe
    FILE LOCKED WITH ONLY READERS, PageProtection: PAGE_EXECUTE
8:50:43.4084390 parent.exe 1076 Process Create ...\.Debug\child1.exe
    PID: 7788, Command line: .\child1.exe
8:50:43.4084474 child1.exe 7788 Process Start
    Parent PID: 1076, Command line: .\child1.exe,
    Current directory: C:\web\OSI\lab2\build\bin\Debug\
```

В событии Process Start потомка указано Parent PID: 1076 (PID родителя) и тот же текущий каталог; блок переменных окружения совпадает с родительским, так как lpEnvironment и lpCurrentDirectory в вызове были равны NULL. Образ parent.exe занимает 0x15000 байт, образы потомков – по 0x12000; поскольку сборка отладочная, помимо ntdll.dll, kernel32.dll и KernelBase.dll каждый процесс загружает ucrtbased.dll и vcruntime140d.dll.

Листинг 10 – Создание child2 (фрагмент трассы Process Monitor, сокращённая запись)

```
8:50:43.4124879 parent.exe 1076 CreateFileMapping ...\.Debug\child2.exe
    FILE LOCKED WITH ONLY READERS, PageProtection: PAGE_EXECUTE
8:50:43.4210807 parent.exe 1076 Process Create ...\.Debug\child2.exe
    PID: 11024, Command line: .\child2.exe
8:50:43.4210924 child2.exe 11024 Process Start
    Parent PID: 1076, Command line: .\child2.exe,
    Current directory: C:\web\OSI\lab2\build\bin\Debug\
```

Child2 создаётся строго после child1 – так же, как в коде parent.c. Обмен данными по каналам в этом сеансе в журнале не отражён: событий WriteFile нет, а единственный ReadFile – страничное чтение ucrtbased.dll (Paging I/O) при завершении child1, а не чтение из канала. По-видимому, это объясняется тем, что данные в этом запуске не вводились (Process Monitor регистрирует операцию по её завершению, поэтому блокирующие ожидания чтения в журнал не попадают). Между 8:50:43.43 и 8:50:56.43 события отсутствуют у всех трёх процессов, то есть они находятся в состоянии ожидания.

Листинг 11 – Завершение по Ctrl+C (фрагмент трассы Process Monitor, сокращённая запись)

```
8:50:56.4314892 child2.exe 11024 Thread Create Thread ID: 11556
8:50:56.4316637 child1.exe 7788 Thread Create Thread ID: 2964
8:50:56.4318282 parent.exe 1076 Thread Create Thread ID: 14048
8:50:56.4324916 child2.exe 11024 Process Exit
    Exit Status: -1073741510
8:50:56.4333246 parent.exe 1076 Process Exit
```

```
Exit Status: -1073741510
8:50:56.4735554 child1.exe 7788 Process Exit
Exit Status: -1073741510
```

В момент нажатия Ctrl+C (8:50:56.431) в течение примерно трети миллисекунды во всех трёх процессах создаётся по новому потоку: консоль доставляет событие CTRL_C_EVENT каждому процессу, подключённому к ней, – и потомкам тоже, поскольку при их создании не использовался флаг CREATE_NEW_PROCESS_GROUP. Все три процесса завершаются с одним кодом -1073741510, что соответствует 0xC000013A (STATUS_CONTROL_C_EXIT) – обработчик по умолчанию, собственный обработчик в программах не устанавливался. Порядок завершения: child2, parent, child1; родитель завершился раньше child1, то есть не дожидался потомков, как это происходит при штатном завершении по концу ввода. Child1 завершается позже остальных (примерно на 41 мс после child2): в его журнале в этот момент видны страничное чтение ucrtbased.dll и загрузка kernel.appcore.dll, msvcrt.dll и rpcrt4.dll – по-видимому, служебная активность обработки сигнала, не относящаяся к логике программы.

Выводы

В ходе работы реализован конвейер из трёх процессов на Win32 API: parent.exe создаёт три анонимных канала и два дочерних процесса с перенаправленными стандартными потоками, child1.exe приводит текст к нижнему регистру, child2.exe убирает задвоенные пробелы, а родитель выводит результат в консоль. Корректность подтверждена тестами из таблицы 4, а трасса Process Monitor подтверждает порядок создания процессов, наследование окружения и каталога и одновременное завершение всех трёх процессов по Ctrl+C. Основная сложность – управление наследованием дескрипторов: если child2 наследует записывающий конец pipe_mid, завершение по концу ввода до него не доходит и родитель зависает; решение – закрывать ненужные концы каналов в родителе до запуска следующего потомка. Изучены функции CreatePipe, SetHandleInformation, CreateProcess с STARTF_USESTDHANDLES, блокирующий обмен через ReadFile и WriteFile и методы анализа поведения процессов в Process Monitor.

Приложение А. Исходный код программы

Листинг А.1 – Файл parent.c

```
#include <windows.h>
#include <stdio.h>
#include <string.h>

#define BUFFER_SIZE 4096

int main(void) {
    HANDLE hPipe1Read, hPipe1Write;    // Parent -> Child1
    HANDLE hPipeMidRead, hPipeMidWrite; // Child1 -> Child2
    HANDLE hPipe2Read, hPipe2Write;    // Child2 -> Parent

    SECURITY_ATTRIBUTES sa;
    sa.nLength = sizeof(SECURITY_ATTRIBUTES);
    sa.bInheritHandle = TRUE;
    sa.lpSecurityDescriptor = NULL;

    if (!CreatePipe(&hPipe1Read, &hPipe1Write, &sa, 0) ||
        !CreatePipe(&hPipeMidRead, &hPipeMidWrite, &sa, 0) ||
        !CreatePipe(&hPipe2Read, &hPipe2Write, &sa, 0)) {
        return 1;
    }

    SetHandleInformation(hPipe1Write, HANDLE_FLAG_INHERIT, 0);
    SetHandleInformation(hPipe2Read, HANDLE_FLAG_INHERIT, 0);

    STARTUPINFOA siChild1;
    PROCESS_INFORMATION piChild1;
    ZeroMemory(&siChild1, sizeof(siChild1));
    ZeroMemory(&piChild1, sizeof(piChild1));
    siChild1.cb = sizeof(siChild1);
    siChild1.hStdInput = hPipe1Read;
    siChild1.hStdOutput = hPipeMidWrite;
    siChild1.hStdError = GetStdHandle(STD_ERROR_HANDLE);
    siChild1.dwFlags |= STARTF_USESTDHANDLES;

    char cmdChild1[] = ".\\child1.exe";
    if (!CreateProcessA(NULL, cmdChild1, NULL, NULL, TRUE, 0, NULL, NULL, &siChild1,
        &piChild1)) {
        return 1;
    }

    CloseHandle(hPipe1Read);
    CloseHandle(hPipeMidWrite);

    STARTUPINFOA siChild2;
    PROCESS_INFORMATION piChild2;
    ZeroMemory(&siChild2, sizeof(siChild2));
    ZeroMemory(&piChild2, sizeof(piChild2));
    siChild2.cb = sizeof(siChild2);
    siChild2.hStdInput = hPipeMidRead;
```

```

    siChild2.hStdOutput = hPipe2Write;
    siChild2.hStdError = GetStdHandle(STD_ERROR_HANDLE);
    siChild2.dwFlags |= STARTF_USESTDHANDLES;

    char cmdChild2[] = ".\\child2.exe";
    if (!CreateProcessA(NULL, cmdChild2, NULL, NULL, TRUE, 0, NULL, NULL, &siChild2,
&piChild2)) {
        return 1;
    }

    CloseHandle(hPipeMidRead);
    CloseHandle(hPipe2Write);

    char input[BUFFER_SIZE];
    DWORD bytesWritten, bytesRead;

    while (1) {
        fflush(stdout);

        if (!fgets(input, sizeof(input), stdin)) {
            break;
        }

        DWORD len = (DWORD)strlen(input);
        if (len == 0 || (len == 1 && input[0] == '\\n')) {
            continue;
        }

        if (!WriteFile(hPipe1Write, input, len, &bytesWritten, NULL)) {
            break;
        }

        char output[BUFFER_SIZE];
        if (ReadFile(hPipe2Read, output, sizeof(output) - 1, &bytesRead, NULL) &&
bytesRead > 0) {
            output[bytesRead] = '\\0';
            printf("%s", output);
            fflush(stdout);
        } else {
            break;
        }
    }

    CloseHandle(hPipe1Write);
    CloseHandle(hPipe2Read);

    WaitForSingleObject(piChild1.hProcess, INFINITE);
    WaitForSingleObject(piChild2.hProcess, INFINITE);

    CloseHandle(piChild1.hProcess);
    CloseHandle(piChild1.hThread);
    CloseHandle(piChild2.hProcess);

```

```

        CloseHandle(piChild2.hThread);

        return 0;
    }

```

Листинг А.2 – Файл child1.c

```

#include <windows.h>
#include <ctype.h>
#define BUFFER_SIZE 4096

int main(void) {
    HANDLE hStdin = GetStdHandle(STD_INPUT_HANDLE);
    HANDLE hStdout = GetStdHandle(STD_OUTPUT_HANDLE);
    char buffer[BUFFER_SIZE];
    DWORD bytesRead, bytesWritten;

    while (ReadFile(hStdin, buffer, BUFFER_SIZE, &bytesRead, NULL) && bytesRead > 0) {
        for (DWORD i = 0; i < bytesRead; i++) {
            buffer[i] = (char)tolower((unsigned char)buffer[i]);
        }
        WriteFile(hStdout, buffer, bytesRead, &bytesWritten, NULL);
    }

    return 0;
}

```

Листинг А.3 – Файл child2.c

```

#include <windows.h>
#include <stdbool.h>

#define BUFFER_SIZE 4096

int main(void) {
    HANDLE hStdin = GetStdHandle(STD_INPUT_HANDLE);
    HANDLE hStdout = GetStdHandle(STD_OUTPUT_HANDLE);
    char inBuffer[BUFFER_SIZE];
    char outBuffer[BUFFER_SIZE];

    DWORD bytesRead, bytesWritten;
    bool prevWasSpace = false;

    while (ReadFile(hStdin, inBuffer, BUFFER_SIZE, &bytesRead, NULL) && bytesRead > 0) {
        DWORD outIdx = 0;

        for (DWORD i = 0; i < bytesRead; i++) {
            char c = inBuffer[i];
            if (c == ' ') {
                if (!prevWasSpace) {
                    outBuffer[outIdx++] = c;
                    prevWasSpace = true;
                }
            }
        }
    }
}

```

```

        }
    } else {
        outBuffer[outIdx++] = c;
        prevWasSpace = false;
    }
}
if (outIdx > 0) {
    WriteFile(hStdout, outBuffer, outIdx, &bytesWritten, NULL);
}
}

return 0;
}

```

Листинг А.4 – Файл CMakeLists.txt

```

cmake_minimum_required(VERSION 3.10)

project(lab2_ipc C)

set(CMAKE_C_STANDARD 11)
set(CMAKE_C_STANDARD_REQUIRED ON)

set(CMAKE_RUNTIME_OUTPUT_DIRECTORY "${CMAKE_BINARY_DIR}/bin")

if(MSVC)
    add_compile_options(/W4)
else()
    add_compile_options(-Wall -Wextra -pedantic)
endif()

add_executable(child1 child1.c)
add_executable(child2 child2.c)
add_executable(parent parent.c)

add_dependencies(parent child1 child2)

```

Приложение Б. Журнал Process Monitor

Приведены события, относящиеся к жизненному циклу процессов и потоков (Process Start/Create/Exit, Thread Create/Exit, Load Image, ReadFile, а также открытие и отображение файлов child1.exe и child2.exe). Не включены обращения к реестру (RegOpenKey, RegQueryValue и др.), служебные запросы к файлам (prefetch, apppatch, QueryBasicInformationFile и т. п.) и блоки переменных окружения, помеченные как [...]. Порядок записей и значения полей сохранены.

Листинг Б.1 – Журнал Process Monitor (parent.exe, child1.exe, child2.exe)

```
8:50:43.3880178 AM parent.exe 1076 Process Start SUCCESS Parent PID: 2364, Command
line: "C:\web\OSI\lab2\build\bin\Debug\parent.exe", Current directory:
C:\web\OSI\lab2\build\bin\Debug\, Environment: [...]
8:50:43.3880340 AM parent.exe 1076 Thread Create SUCCESS Thread ID: 11020
8:50:43.3896838 AM parent.exe 1076 Load Image
C:\web\OSI\lab2\build\bin\Debug\parent.exe SUCCESS Image Base: 0x7ff796650000, Image
Size: 0x15000
8:50:43.3897994 AM parent.exe 1076 Load Image C:\Windows\System32\ntdll.dll SUCCESS
Image Base: 0x7ff93d6d0000, Image Size: 0x1f8000
8:50:43.3909741 AM parent.exe 1076 Load Image C:\Windows\System32\kernel32.dll
SUCCESS Image Base: 0x7ff93ca20000, Image Size: 0xc2000
8:50:43.3912058 AM parent.exe 1076 Load Image C:\Windows\System32\KernelBase.dll
SUCCESS Image Base: 0x7ff93b1c0000, Image Size: 0x2f6000
8:50:43.3937286 AM parent.exe 1076 Thread Create SUCCESS Thread ID: 2380
8:50:43.3938628 AM parent.exe 1076 Thread Create SUCCESS Thread ID: 2616
8:50:43.3958353 AM parent.exe 1076 Load Image C:\Windows\System32\vcruntime140d.dll
SUCCESS Image Base: 0x7ff90c820000, Image Size: 0x30000
8:50:43.3961431 AM parent.exe 1076 Load Image C:\Windows\System32\ucrtbased.dll
SUCCESS Image Base: 0x7ff8db810000, Image Size: 0x204000
8:50:43.3962618 AM parent.exe 1076 Thread Create SUCCESS Thread ID: 12408
8:50:43.3989077 AM parent.exe 1076 CreateFile
C:\web\OSI\lab2\build\bin\Debug\child1.exe SUCCESS Desired Access: Read Attributes,
Disposition: Open, Options: Open Reparse Point, Attributes: n/a, ShareMode: Read, Write,
Delete, AllocationSize: n/a, OpenResult: Opened
8:50:43.3992324 AM parent.exe 1076 CreateFile
C:\web\OSI\lab2\build\bin\Debug\child1.exe SUCCESS Desired Access: Read Attributes,
Disposition: Open, Options: Open Reparse Point, Attributes: n/a, ShareMode: Read, Write,
Delete, AllocationSize: n/a, OpenResult: Opened
8:50:43.3995045 AM parent.exe 1076 Load Image C:\Windows\System32\sechost.dll
SUCCESS Image Base: 0x7ff93b6f0000, Image Size: 0x9f000
8:50:43.3997341 AM parent.exe 1076 Load Image C:\Windows\System32\rpcrt4.dll
SUCCESS Image Base: 0x7ff93d020000, Image Size: 0x120000
8:50:43.3999688 AM parent.exe 1076 Load Image C:\Windows\System32\bcrypt.dll
SUCCESS Image Base: 0x7ff93ade0000, Image Size: 0x27000
8:50:43.4007316 AM parent.exe 1076 CreateFile
C:\web\OSI\lab2\build\bin\Debug\child1.exe SUCCESS Desired Access: Read Data/List
Directory, Execute/Traverse, Read Attributes, Synchronize, Disposition: Open, Options:
Synchronous IO Non-Alert, Non-Directory File, Attributes: N, ShareMode: Read, Delete,
AllocationSize: n/a, OpenResult: Opened
8:50:43.4007867 AM parent.exe 1076 CreateFileMapping
C:\web\OSI\lab2\build\bin\Debug\child1.exe FILE LOCKED WITH ONLY READERS SyncType:
SyncTypeCreateSection, PageProtection: PAGE_EXECUTE
8:50:43.4008232 AM parent.exe 1076 CreateFileMapping
C:\web\OSI\lab2\build\bin\Debug\child1.exe SUCCESS SyncType: SyncTypeOther
```

```

8:50:43.4084390 AM parent.exe 1076 Process Create
C:\web\OSI\lab2\build\bin\Debug\child1.exe SUCCESS PID: 7788, Command line:
.\child1.exe
8:50:43.4084474 AM child1.exe 7788 Process Start SUCCESS Parent PID: 1076, Command
line: .\child1.exe, Current directory: C:\web\OSI\lab2\build\bin\Debug\, Environment:
[...]
8:50:43.4084583 AM child1.exe 7788 Thread Create SUCCESS Thread ID: 8816
8:50:43.4114536 AM child1.exe 7788 Load Image
C:\web\OSI\lab2\build\bin\Debug\child1.exe SUCCESS Image Base: 0x7ff6d7a00000, Image
Size: 0x12000
8:50:43.4115450 AM child1.exe 7788 Load Image C:\Windows\System32\ntdll.dll SUCCESS
Image Base: 0x7ff93d6d0000, Image Size: 0x1f8000
8:50:43.4117276 AM parent.exe 1076 CreateFile
C:\web\OSI\lab2\build\bin\Debug\child2.exe SUCCESS Desired Access: Read Attributes,
Disposition: Open, Options: Open Reparse Point, Attributes: n/a, ShareMode: Read, Write,
Delete, AllocationSize: n/a, OpenResult: Opened
8:50:43.4121167 AM parent.exe 1076 CreateFile
C:\web\OSI\lab2\build\bin\Debug\child2.exe SUCCESS Desired Access: Read Attributes,
Disposition: Open, Options: Open Reparse Point, Attributes: n/a, ShareMode: Read, Write,
Delete, AllocationSize: n/a, OpenResult: Opened
8:50:43.4124296 AM parent.exe 1076 CreateFile
C:\web\OSI\lab2\build\bin\Debug\child2.exe SUCCESS Desired Access: Read Data/List
Directory, Execute/Traverse, Read Attributes, Synchronize, Disposition: Open, Options:
Synchronous IO Non-Alert, Non-Directory File, Attributes: N, ShareMode: Read, Delete,
AllocationSize: n/a, OpenResult: Opened
8:50:43.4124655 AM child1.exe 7788 Load Image C:\Windows\System32\kernel32.dll
SUCCESS Image Base: 0x7ff93ca20000, Image Size: 0xc2000
8:50:43.4124879 AM parent.exe 1076 CreateFileMapping
C:\web\OSI\lab2\build\bin\Debug\child2.exe FILE LOCKED WITH ONLY READERS SyncType:
SyncTypeCreateSection, PageProtection: PAGE_EXECUTE
8:50:43.4125347 AM parent.exe 1076 CreateFileMapping
C:\web\OSI\lab2\build\bin\Debug\child2.exe SUCCESS SyncType: SyncTypeOther
8:50:43.4126803 AM child1.exe 7788 Load Image C:\Windows\System32\KernelBase.dll
SUCCESS Image Base: 0x7ff93b1c0000, Image Size: 0x2f6000
8:50:43.4150493 AM child1.exe 7788 Thread Create SUCCESS Thread ID: 11120
8:50:43.4151754 AM child1.exe 7788 Thread Create SUCCESS Thread ID: 14820
8:50:43.4163668 AM child1.exe 7788 Load Image C:\Windows\System32\ucrtbased.dll
SUCCESS Image Base: 0x7ff8db810000, Image Size: 0x204000
8:50:43.4187986 AM child1.exe 7788 Load Image C:\Windows\System32\vcruntime140d.dll
SUCCESS Image Base: 0x7ff90c820000, Image Size: 0x30000
8:50:43.4210807 AM parent.exe 1076 Process Create
C:\web\OSI\lab2\build\bin\Debug\child2.exe SUCCESS PID: 11024, Command line:
.\child2.exe
8:50:43.4210924 AM child2.exe 11024 Process Start SUCCESS Parent PID: 1076, Command
line: .\child2.exe, Current directory: C:\web\OSI\lab2\build\bin\Debug\, Environment:
[...]
8:50:43.4211063 AM child2.exe 11024 Thread Create SUCCESS Thread ID: 2384
8:50:43.4227934 AM child2.exe 11024 Load Image
C:\web\OSI\lab2\build\bin\Debug\child2.exe SUCCESS Image Base: 0x7ff673040000, Image
Size: 0x12000
8:50:43.4229060 AM child2.exe 11024 Load Image C:\Windows\System32\ntdll.dll
SUCCESS Image Base: 0x7ff93d6d0000, Image Size: 0x1f8000

```

```

8:50:43.4240230 AM child2.exe 11024 Load Image C:\Windows\System32\kernel32.dll
SUCCESS Image Base: 0x7ff93ca20000, Image Size: 0xc2000
8:50:43.4243053 AM child2.exe 11024 Load Image C:\Windows\System32\KernelBase.dll
SUCCESS Image Base: 0x7ff93b1c0000, Image Size: 0x2f6000
8:50:43.4263937 AM child2.exe 11024 Thread Create SUCCESS Thread ID: 2768
8:50:43.4265308 AM child2.exe 11024 Thread Create SUCCESS Thread ID: 12264
8:50:43.4276724 AM child2.exe 11024 Load Image C:\Windows\System32\ucrtbased.dll
SUCCESS Image Base: 0x7ff8db810000, Image Size: 0x204000
8:50:43.4279251 AM child2.exe 11024 Load Image C:\Windows\System32\vcruntime140d.dll
SUCCESS Image Base: 0x7ff90c820000, Image Size: 0x30000
8:50:56.4314892 AM child2.exe 11024 Thread Create SUCCESS Thread ID: 11556
8:50:56.4316637 AM child1.exe 7788 Thread Create SUCCESS Thread ID: 2964
8:50:56.4317577 AM child2.exe 11024 Thread Exit SUCCESS Thread ID: 2768, User Time:
0.0000000, Kernel Time: 0.0000000
8:50:56.4317643 AM child2.exe 11024 Thread Exit SUCCESS Thread ID: 2384, User Time:
0.0000000, Kernel Time: 0.0000000
8:50:56.4317724 AM child2.exe 11024 Thread Exit SUCCESS Thread ID: 12264, User
Time: 0.0000000, Kernel Time: 0.0000000
8:50:56.4318282 AM parent.exe 1076 Thread Create SUCCESS Thread ID: 14048
8:50:56.4319681 AM child2.exe 11024 Thread Exit SUCCESS Thread ID: 11556, User
Time: 0.0000000, Kernel Time: 0.0000000
8:50:56.4323446 AM parent.exe 1076 Thread Exit SUCCESS Thread ID: 11020, User Time:
0.0000000, Kernel Time: 0.0000000
8:50:56.4324292 AM parent.exe 1076 Thread Exit SUCCESS Thread ID: 12408, User Time:
0.0000000, Kernel Time: 0.0000000
8:50:56.4324583 AM parent.exe 1076 Thread Exit SUCCESS Thread ID: 2616, User Time:
0.0000000, Kernel Time: 0.0000000
8:50:56.4324916 AM child2.exe 11024 Process Exit SUCCESS Exit Status: -1073741510,
User Time: 0.0000000 seconds, Kernel Time: 0.0000000 seconds, Private Bytes: 589,824,
Peak Private Bytes: 598,016, Working Set: 3,387,392, Peak Working Set: 3,391,488
8:50:56.4324985 AM parent.exe 1076 Thread Exit SUCCESS Thread ID: 14048, User Time:
0.0000000, Kernel Time: 0.0000000
8:50:56.4325015 AM parent.exe 1076 Thread Exit SUCCESS Thread ID: 2380, User Time:
0.0000000, Kernel Time: 0.0000000
8:50:56.4329290 AM child1.exe 7788 Load Image C:\Windows\System32\kernel.appcore.dll
SUCCESS Image Base: 0x7ff939570000, Image Size: 0x12000
8:50:56.4332564 AM child1.exe 7788 Load Image C:\Windows\System32\msvcrt.dll
SUCCESS Image Base: 0x7ff93d370000, Image Size: 0x9e000
8:50:56.4333246 AM parent.exe 1076 Process Exit SUCCESS Exit Status: -1073741510,
User Time: 0.0000000 seconds, Kernel Time: 0.0000000 seconds, Private Bytes: 765,952,
Peak Private Bytes: 774,144, Working Set: 4,022,272, Peak Working Set: 4,026,368
8:50:56.4336763 AM child1.exe 7788 Load Image C:\Windows\System32\rpcrt4.dll
SUCCESS Image Base: 0x7ff93d020000, Image Size: 0x120000
8:50:56.4352941 AM child1.exe 7788 ReadFile C:\Windows\System32\ucrtbased.dll
SUCCESS Offset: 1,687,552, Length: 16,384, I/O Flags: Non-cached, Paging I/O,
Synchronous Paging I/O, Priority: Normal
8:50:56.4353895 AM child1.exe 7788 Thread Exit SUCCESS Thread ID: 14820, User Time:
0.0000000, Kernel Time: 0.0000000
8:50:56.4354547 AM child1.exe 7788 Thread Exit SUCCESS Thread ID: 11120, User Time:
0.0000000, Kernel Time: 0.0000000
8:50:56.4355882 AM child1.exe 7788 Thread Exit SUCCESS Thread ID: 2964, User Time:
0.0000000, Kernel Time: 0.0000000

```

```
8:50:56.4729622 AM child1.exe 7788 Thread Exit SUCCESS Thread ID: 8816, User Time:
0.0000000, Kernel Time: 0.0000000
8:50:56.4735554 AM child1.exe 7788 Process Exit SUCCESS Exit Status: -1073741510,
User Time: 0.0000000 seconds, Kernel Time: 0.0000000 seconds, Private Bytes: 737,280,
Peak Private Bytes: 745,472, Working Set: 3,833,856, Peak Working Set: 3,837,952
```